```
> library(knitr)
> opts_chunk$set(
+   prompt = FALSE,
+   comment = NA,
+   highlight = TRUE,
+   background = '#F7F7F7',
+   tidy = FALSE,
+   fig.align = 'center',
+   out.width = '0.7\\textwidth'
+ )
```

# EdgeCount: Statistical Theory and Algorithmic Efficiency

Joel R. Pradines

December 16, 2025

**Abstract**

This vignette details the statistical framework underlying the `EdgeCount` package. First, the Random Graph with Given Expected Degrees (RGGED) model is described. Second, the derivation of edge-count statistics is presented. Third, the algebraic optimizations that allow the package to calculate connectedness statistics with linear time complexity are explained. Last, the implementation of vectorized functions contained in the package is discussed, and it is shown that they have optimal time complexity.

## Contents



## 1 Introduction

The `EdgeCount` package provides methods to analyze the connectedness of vertex sets in large, sparse, undirected graphs. Whether analyzing interaction networks or bipartite group-element graphs, the question is:

*Is an observed number of edges, called edge count, within a set (or between two sets) significantly higher than expected by chance?*

To answer this, two components must be implemented.

First, the random experiment defining the concept of *chance* must be clearly specified and, in general, simulated to provide a control distribution.

Second, one needs to be able to fairly compare edge-count values to each other. A fair comparison requires accounting for vertex degrees, as these may vary by orders of magnitude. This is achieved here by using a null model of random graphs that preserves the expected degrees of vertices [5, 6]. Specifically, the average degree of a vertex over all random graph realizations matches its observed value in the original network. The analytical expressions for the likelihoods of edge-count variables in this model were derived in [15]. These expressions enable fair and fast comparisons of edge-count values.

## 2 Random graph model

Let $G(V, E)$ be an undirected graph with no loops and no multiple edges.

- Let $N = |V|$ be the order of the graph (number of vertices).
- Let $M = |E|$ be the size of the graph (number of edges).
- Let $k_i$ be the degree of vertex $v_i$.
- Let $\langle k \rangle$ be the the average vertex degree.

Call the degree sequence of $G$ the nonincreasing sequence of all its vertex degrees. An obvious null model for $G$ is that of a random graph whose realizations always preserve the degree sequence. This model is referred to as the Random Graph with Fixed Degree Sequence (RGFDS). However, it is known that the analytical treatment of the RGFDS is a difficult problem [4, 13, 7, 14]. Namely, deriving simple analytical expressions for the likelihoods of edge-count variables is in general hard with the RGFDS. This implies that slow numerical methods based on simulations must be used [10, 12, 11].

A less constrained random graph model than the RGFDS is the Random Graph with Given Expected Degrees (RGGED) [5, 6]. While an individual realization of the RGGED might not preserve a vertex degree, it is preserved on average over all realizations. The key feature of the RGGED is that edges are treated as independent random variables, enabling the derivation of analytical expressions for the distributions of edge-count variables [15].

In the RGGED model, edges are treated as independent Bernoulli random variables: an edge $v_i v_j$ exists with probability $p_{ij}$ (Bernoulli parameter) and does not exist with probability $1 - p_{ij}$. When the graph is large and sparse on average over its vertices ($\langle k \rangle^2 \ll 2M$), the probability $p_{ij}$ of observing an edge between vertices $v_i$ and $v_j$ is given by [15]:

$$p_{ij} \simeq 1 - \exp\left(-k_i k_j / (2M)\right). \tag{1}$$

Useful upper bounds for $p_{ij}$ are

$$p_{ij} \leq \min\left(1, \frac{k_i k_j}{2M}\right) \leq \frac{k_i k_j}{2M}. \tag{2}$$

### 2.1 Within and between variables

The `EdgeCount` package considers two types of edge-count variables:

- "within" for the number $z$ of edges observed in a given set $S$ of vertices
- "between" for the number $z$ of edges observed between two sets $S_1$ and $S_2$ of vertices

Observed values $z$ are obviously dependent on set sizes, but they should also be made conditional to degrees of the vertices in the sets. Hence, to an observed value $z$ is associated a corresponding random variable $Z$ in the RGGED.

### 2.2 Edge-count distribution

Edge-count random variables in the RGGED model of a large and sparse graph have approximate Poisson distribution. This is because they are the sum of independent Bernoulli random variables with small parameters $p_{ij} \ll 1$, even when these parameters have different values [9, 8, 15].

Call $Z$ an edge-count random variable defined by the sum of $m$ potential edges, each edge corresponding to a pair $v_i v_j$ of vertices:

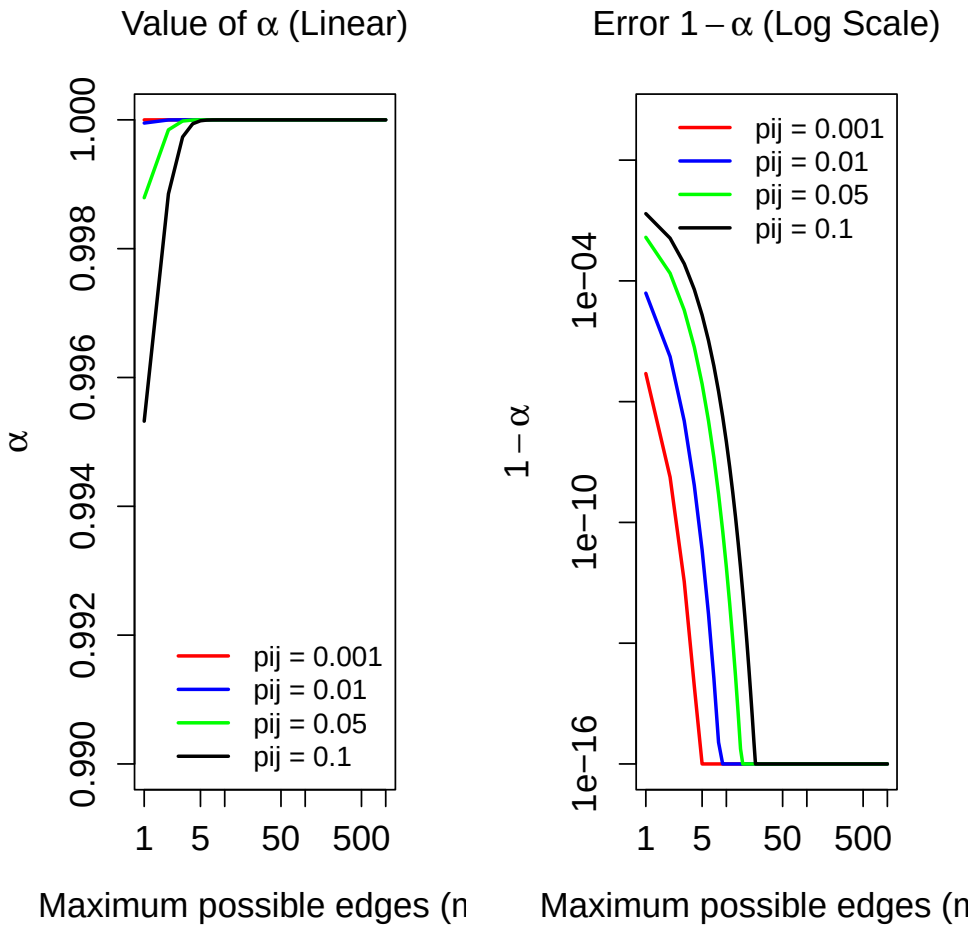
$$Z = \sum_{u=1}^{m} B_u \quad \text{with} \quad B_u \sim \text{Bernoulli}\left(p_u = p_{ij}\right). \tag{3}$$

Then, we have $Z \sim \text{Poisson}\left(\lambda, m\right)$ as defined by

$$\Pr(Z = z) = \frac{e^{-\lambda}}{\alpha}\frac{\lambda^z}{z!} \quad \text{with} \quad \lambda = \sum_{u=1}^{m} p_u \quad \text{and} \quad \alpha = e^{-\lambda}\sum_{x=0}^{m}\frac{\lambda^x}{x!}. \tag{4}$$

The normalization factor $\alpha$ is utilized because an observed number $z$ of edges is bounded by a maximum number $m$ of edges. This said, we have $\alpha \simeq 1$ when $p_{ij} \ll 1$. This is numerically illustrated next.

```
> # Function to calculate alpha and its complement
> get_alpha_stats <- function(m, pij_avg) {
+   lambda <- m * pij_avg
+   alpha <- stats::ppois(m, lambda)
+   alpha_comp <- stats::ppois(m, lambda, lower.tail = FALSE)
+   return(c(alpha, alpha_comp))
+ }
> # Parameters
> pij_values <- c(0.001, 0.01, 0.05, 0.1)
> m_values <- c(1:29,seq(30, 1000, by = 10))
> colors <- c("red", "blue", "green", "black")
> # Plots
> par(mfrow=c(1, 2), mar=c(5, 5, 4, 2) + 0.1, cex.lab=1.25,
+     cex.axis=1.25, cex.main=1.25)
> plot(1, type="n", xlab="Maximum possible edges (m)",
+      ylab=expression(alpha),
+      xlim=c(1, 1000), ylim=c(0.99, 1.00),
+      log="x",
+      main=expression(paste("Value of ", alpha, " (Linear)")))
> for(i in seq_along(pij_values)) {
+   lambdas <- m_values * pij_values[i]
+   y_vals <- ppois(m_values, lambdas)
+   lines(m_values, y_vals, col=colors[i], lwd=2)
+ }
> legend("bottomright", legend=paste("pij =", pij_values),
+        col=colors, lwd=2, bty="n", cex=1)
> epsilon <- 1e-16
> plot(1, type="n", xlab="Maximum possible edges (m)",
+      ylab=expression(1 - alpha),
+      xlim=c(1, 1000), ylim=c(epsilon, 1),
+      log="xy",
+      main=expression(paste("Error ", 1 - alpha, " (Log Scale)")))
> for(i in seq_along(pij_values)) {
+   lambdas <- m_values * pij_values[i]
+   y_vals <- ppois(m_values, lambdas, lower.tail = FALSE)
+   y_vals <- pmax(y_vals, epsilon)
+   lines(m_values, y_vals, col=colors[i], lwd=2)
+ }
> legend("topright", legend=paste("pij =", pij_values),
+        col=colors, lwd=2, bty="n", cex=1)
```




For a within variable associated with a set $S$ we have

$$m = |S|\left(|S|-1\right)/2 \quad \text{and} \quad \lambda = \sum_{i<j,\in S} p_{ij}. \tag{5}$$

For a between variable associated with two sets $S_1$ and $S_2$ we have

$$m = |S_1|\,|S_2| \quad \text{and} \quad \lambda = \sum_{i\in S_1, j\in S_2} p_{ij}. \tag{6}$$

Since the distribution of an edge-count variable is easily computed, p-values are also calculated.

$$\Pr\left(Z \geq z\right) = \sum_{x=z}^{m} \Pr\left(Z = x\right). \tag{7}$$

P-values are calculated with `R` function `ppois()`.

```
> alpha <- stats::ppois(m, lambda, lower.tail = TRUE)
> p_value <- (stats::ppois(z - 1, lambda, lower.tail = FALSE) -
+               stats::ppois(m, lambda, lower.tail = FALSE)) / alpha
```

The Poisson distribution enables quantification of the variability on vertex degree modeled by the RGGED. In the model, the degree of a vertex $v_i$ with initial degree $k_i$ is a random variable $K_i$ defined by the edge

count between this vertex and all the others:

$$\lambda_i = \frac{k_i}{2M} \sum_{j \neq i} k_j \simeq k_i. \tag{8}$$

That is, $K_i$ has mean $k_i$ and standard deviation $\sqrt{k_i}$. The introduced relative variability on $k_i$ is reasonable: $1/\sqrt{k_i}$.

### 2.3 Edge-count effect size

As explained earlier, the RGGED is not directly utilized as a null model to answer a question. Rather, it is used as an intermediate step that enables fair and fast comparison of diverse observed edge counts. Such a comparison can be between edge-count variables defined by sets of different sizes. In this case, an effect size is the most appropriate quantity for comparison.

Because edge-count random variables have Poisson distribution, the utilized effect size is

$$\log_2 \text{Anscombe ratio:} \quad lAr\,(z, \lambda) = \frac{1}{2} \log_2 \frac{z + 3/8}{\lambda + 3/8}. \tag{9}$$

The addition of $3/8$ stems from the work of Anscombe [2], who showed that $\sqrt{X + 3/8}$ is the optimal variance-stabilizing transformation for a Poisson random variable $X$.

## 3 Fast methods for sparse interaction networks

All the code for representation and analysis of undirected unipartite graphs (interaction networks) is contained in S4 clasees `ECGraph` and `ECProb`.

### 3.1 Sparseness condition

A condition for sparseness in the `EdgeCount` framework is

$$\max_{i,j} \frac{k_i k_j}{2M} < 1. \tag{10}$$

This sparseness condition can be tested with method `summarize_suitability_fast()` of class `ECProb`. The method returns a list containing the proportion `pij_over_1`, among all possible pairs $v_i, v_j$ of vertices, of pairs such that ve$(k_i k_j)/(2M) \geq 1$. An ideally sparse graph yields `pij_over_1` $= 0$.

Now, approximating $p_{ij}$ values with $k_i k_j/(2M)$ is a conservative approach in the RGGED. Namely, using this approximation

- overestimates $p_{ij}$ values,
- overestimates lambda values,
- overestimates edge-count p-values,
- underestimates effect sizes.

Therefore, utilizing this approximation by default is a reasonable approach. The approximation is called "fast approximation" because it enables faster calculation of edge-count statistics. The speedup is obtained in the computation of lambda parameters.

### 3.2 Within-set connectedness

Consider a vertex set $S$ of size $|S| = n$. With the fast approximation the $\lambda$ parameter is

$$\lambda_{in} = \sum_{i<j,\in S} \frac{k_i k_j}{2M}. \tag{11}$$

A naive implementation would have time complexity $\mathcal{O}\left(n^2\right)$. Recall the expression for the square of a sum

$$\left(\sum_{i\in S} k_i\right)^2 = \sum_{i\in S} k_i^2 + 2\sum_{i<j} k_i k_j. \tag{12}$$

Rearranging the cross terms

$$\sum_{i<j} k_i k_j = \frac{1}{2}\left[\left(\sum_{i\in S} k_i\right)^2 - \sum_{i\in S} k_i^2\right], \tag{13}$$

and substituting in the expression for lambda yields

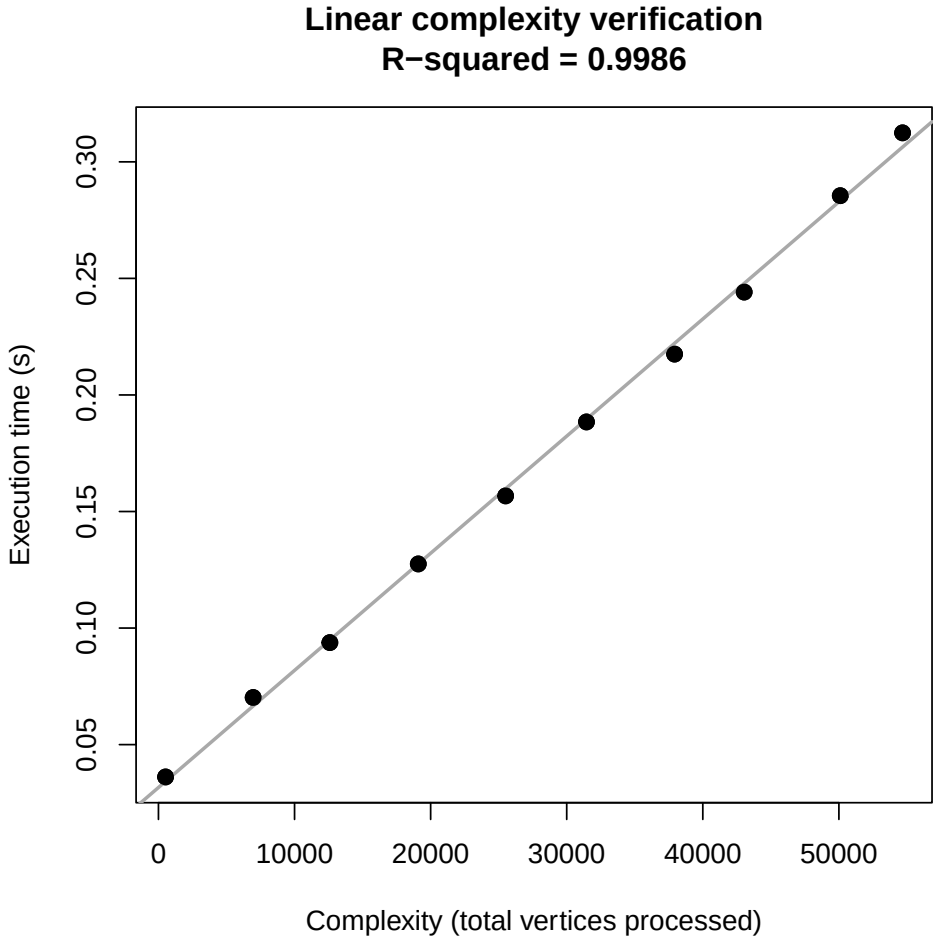
$$\lambda_{in} = \frac{1}{4M}\left[\left(\sum_{i\in S} k_i\right)^2 - \sum_{i\in S} k_i^2\right]. \tag{14}$$

With this expression the computation of $\lambda_{in}$ has now linear time complexity: $\mathcal{O}(n)$. Method `edge_count_statistics()` of class `ECProb` when used with argument `lambda_method = "fast"` implements this linear computation. Method `calculate_in_stats_fast_vectorized()` is an implementation of "within" edge count statistics for multiple sets provided as input. This method is fully vectorized and uses the `data.table` package [3]. Here is a numerical illustration that the implementation achieves optimal, linear here, time complexity.[1] The illustration is based on the interaction network `sample_ecg` contained in the `EdgeCount` package and derived from the STRING database [16, 1].

```
> library(data.table)
> library(EdgeCount)
> data("sample_ecg")
> ecprob <- ECProb(sample_ecg)
> n_sim <- 5
> n_collections <- 10
> collection_size_universe <- seq(10, 1000, length.out = n_collections)
> set_size_universe <- 10:100
> set.seed(1)
> get_collection <- function(target_n_sets){
+   n_sets <- round(target_n_sets)
+   sets <- lapply(1:n_sets, function(x){
+     set_size <- sample(set_size_universe, 1)
+     sample(ecprob@names, set_size)
+   })
+   set_membership_dt <- data.table(
+     set_id = rep(seq_along(sets), lengths(sets)),
+     element = unlist(sets)
+   )
+   return(set_membership_dt)
+ }
> collections <- lapply(collection_size_universe, get_collection)
```

[1] Such numerical results are always dependent on the utilized CPU. A faster machine might require larger simulated sets to evidence linearity.

```
> results_list <- lapply(collections, function(set_membership_dt){
+   sets_dt <- data.table(set_id = unique(set_membership_dt$set_id))
+   times <- numeric(n_sim)
+   res <- NULL
+   for(k in 1:n_sim){
+     start_time <- Sys.time()
+     capture.output(
+       res <- calculate_in_stats_fast_vectorized(ecprob, sets_dt, set_membership_dt)
+     )
+     end_time <- Sys.time()
+     times[k] <- as.numeric(end_time - start_time, units = "secs")
+   }
+   complexity <- sum(res$set_size, na.rm = TRUE)
+   data.table(complexity = complexity, time = min(times))
+ })
> results_dt <- rbindlist(results_list)
> lm_model <- lm(time ~ complexity, data = results_dt)
> r_sq <- summary(lm_model)$r.squared
> plot(results_dt$complexity, results_dt$time, pch=19, cex=1.2,
+      xlab = "Complexity (total vertices processed)",
+      ylab = "Execution time (s)",
+      main = paste("Linear complexity verification\nR-squared =", round(r_sq, 4)))
> abline(lm_model, col = "darkgrey", lwd=2)
> points(results_dt$complexity, results_dt$time, pch=19, cex=1.2)
```




The execution time scales linearly (R-squared value close to 1) with the optimal time complexity, which is the sum of the set sizes in each set collection used as input.

The fast approximation of $p_{ij}$ also enables optimal computation of "between" connectedness statistics.

### 3.3 Between-sets connectedness

Consider two disjoint vertex sets $S_1$ and $S_2$ with sizes $n_1$ and $n_2$. With the fast approximation, the $\lambda$ parameter is

$$\lambda_{btw} = \sum_{i \in S_1, j \in S_2} \frac{k_i k_j}{2M}. \tag{15}$$

A naive implementation summing over all pairs would have time complexity $\mathcal{O}(n_1 n_2)$. However, the expression can be factored as the product of two independent sums:

$$\lambda_{btw} = \frac{1}{2M} \left( \sum_{i \in S_1} k_i \right) \left( \sum_{j \in S_2} k_j \right). \tag{16}$$

This factorization reduces the calculation to simply summing the degrees within each set and multiplying the results. Consequently, the computation of $\lambda_{btw}$ has linear time complexity: $\mathcal{O}(n_1 + n_2)$.

Fast calculation of 'between' edge-count statistics for a single pair of sets is implemented in method `edge_count_statistics()` of class `ECProb` by setting both arguments `set1` and `set2` to non-null vectors

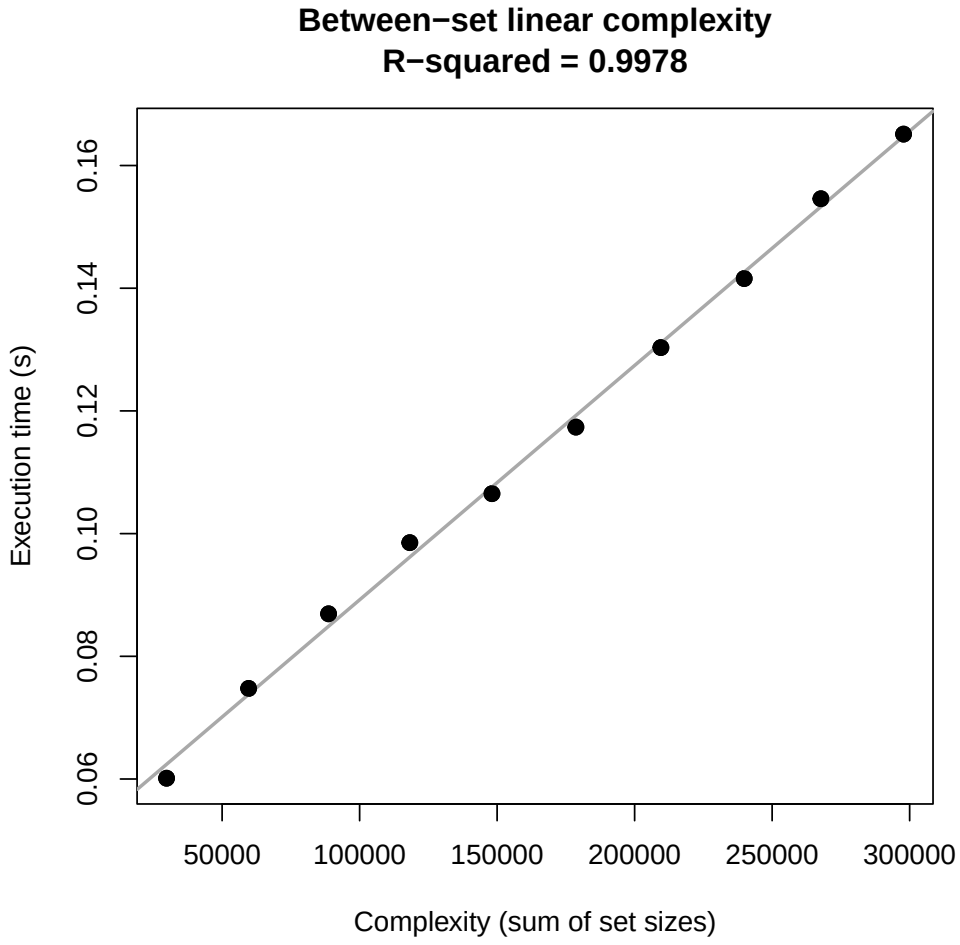
and by using `lambda_method = "fast"`.

For multiple pairs of sets, the appropriate method is `calculate_between_stats_fast_vectorized()`. It implements the fast approximation and is fully vectorized in parts by using the `data.table` package [3]. The following simulation confirms that it achieves optimal, linear here, time complexity.[2]

```
> library(data.table)
> library(EdgeCount)
> data("sample_ecg")
> ecprob <- ECProb(sample_ecg)
> n_sim <- 10
> n_collections <- 10
> n_pairs_universe <- seq(500, 5000, length.out = n_collections)
> set_size_universe <- 10:50
> set.seed(123)
> get_between_collection <- function(n_pairs){
+
+   n_pairs <- round(n_pairs)
+   total_sets <- 2 * n_pairs
+
+   sets <- lapply(1:total_sets, function(x) {
+     sample(ecprob@names, sample(set_size_universe, 1))
+   })
+
+   set_membership_dt <- data.table(
+     set_id = rep(seq_along(sets), lengths(sets)),
+     element = unlist(sets)
+   )
+
+   pairs_dt <- data.table(
+     set1 = seq(1, total_sets, by = 2),
+     set2 = seq(2, total_sets, by = 2)
+   )
+
+   return(list(pairs = pairs_dt, membership = set_membership_dt))
+ }
> collections_btw <- lapply(n_pairs_universe, get_between_collection)
> results_list_btw <- lapply(collections_btw, function(input_data){
+
+   pairs_dt <- input_data$pairs
+   membership_dt <- input_data$membership
+
+   times <- numeric(n_sim)
+   res <- NULL
+
+   for(k in 1:n_sim){
+     start_time <- Sys.time()
+     temp_res <- NULL
+     capture.output(
+       temp_res <- calculate_between_stats_fast_vectorized(ecprob, pairs_dt, membership_dt)
+     )
+     res <- temp_res
+     end_time <- Sys.time()
```

[2]Such numerical results are always dependent on the utilized CPU. A faster machine might require larger simulated sets to evidence linearity.

```
+     times[k] <- as.numeric(end_time - start_time, units = "secs")
+   }
+
+   complexity <- sum(res$size1, na.rm = TRUE) + sum(res$size2, na.rm = TRUE)
+
+   data.table(complexity = complexity, time = min(times))
+ })
> results_dt_btw <- rbindlist(results_list_btw)
> lm_model_btw <- lm(time ~ complexity, data = results_dt_btw)
> r_sq_btw <- summary(lm_model_btw)$r.squared
> plot(results_dt_btw$complexity, results_dt_btw$time, pch=19, cex=1.2,
+      xlab = "Complexity (sum of set sizes)",
+      ylab = "Execution time (s)",
+      main = paste("Between-set linear complexity\nR-squared =", round(r_sq_btw, 4)))
> abline(lm_model_btw, col = "darkgrey", lwd=2)
> points(results_dt_btw$complexity, results_dt_btw$time, pch=19, cex=1.2)
```




# 4 Fast methods for sparse bipartite networks

Class `ECTermScoring` complements the functionality of `ECGraph` and `ECProb` to the case of undirected bipartite graphs. These represent here membership of elements within groups, called terms in the package.

### 4.1 Sparseness condition

Call $\mathcal{T}$ the set of vertices that represent terms and $\mathcal{E}$ the set of elements. The sparseness condition is:

$$\max_{i \in \mathcal{T}, j \in \mathcal{E}} \frac{k_i k_j}{2M} < 1. \tag{17}$$

This is the sparseness condition already presented for interaction networks, but adapted to edges existing only between terms and elements. Method `summarize_suitability_fast()` of class `ECTermScoring` is used to test this condition. The method returns a list containing the proportion `pij_over_1`, among all possible pairs $v_i, v_j$, of values $(k_i k_j)/(2M) \geq 1$. An ideally sparse graph yields `pij_over_1` $= 0$.

But as mentioned for interaction networks, approximating $p_{ij}$ values with $k_i k_j/(2M)$ is a conservative approach in the RGGED. Namely, using this approximation

- overestimates $p_{ij}$ values,
- overestimates lambda values,
- overestimates edge-count p-values,
- underestimates effect sizes.

Therefore, utilizing this approximation by default is a reasonable approach.

### 4.2 Fast approximation

The fast approximation implemented in `ECTermScoring` is the same used in `ECProb` for between connectedeness, but applied to edge counts between elements and terms.

## References


[1] Dataset: sample_ecg. `sample_ecg` is a subset of the human STRING network (version 12.0) with interaction scores of 900 or higher. Vertex IDs are Entrez Gene IDs (see Carlson, 2019). The network has been trimmed to create a smaller, more manageable example for the package's vignettes and tests. Source: string-db.org. Data from STRING is licensed under CC BY 4.0., 2025.

[2] Francis J. Anscombe. The transformation of poisson, binomial and negative-binomial data. *Biometrika*, 35(3/4):246–254, 1948.

[3] T Barrett, M Dowle, A Srinivasan, J Gorecki, M Chirico, T Hocking, B Schwendinger, and I Krylov. *data.table: Extension of 'data.frame'*, 2025. R package version 1.17.99.

[4] Edward A Bender and E Rodney Canfield. The asymptotic number of labelled graphs with given degree sequences. *Journal of Combinatorial Theory, Series A*, 24(3):296–307, 1978.

[5] Fan Chung and Linyuan Lu. Connected components in random graphs with given expected degree sequences. *Annals of Combinatorics*, 6(2):125–145, 2002.

[6] Fan R. K. Chung and Linyuan Lu. The average distances in random graphs with given expected degrees. *Proceedings of the National Academy of Sciences of the United States of America*, 99:15879–15882, 2002.

[7] S Itzkovitz, R Milo, N Kashtan, G Ziv, and U Alon. Subgraphs in random networks. *Physical Review E*, 68(2):026127, 2003.

[8] Johannes Kerstan. Verallgemeinerung eines satzes von prochorow und le cam. *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, 2(3):173–179, 1964.

[9] Lucien Le Cam. An approximation theorem for the poisson binomial distribution. *Pacific Journal of Mathematics*, 10(4):1181–1197, 1960.

[10] Sergei Maslov and Kim Sneppen. Specificity and stability in topology of protein networks. *Science*, 296(5569):910–913, 2002.

[11] R Milo, N Kashtan, S Itzkovitz, MEJ Newman, and U Alon. Uniform generation of random graphs with arbitrary degree sequences. *arXiv preprint cond-mat/0312028*, 2003.

[12] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. Network motifs: simple building blocks of complex networks. *Science*, 298(5594):824–827, 2002.

[13] Michael Molloy and Bruce Reed. A critical point for random graphs with a given degree sequence. *Random Structures & Algorithms*, 6(2-3):161–179, 1995.

[14] Juyong Park and M. E. J. Newman. Statistical mechanics of networks. *Physical Review E*, 70(6):066117, 2004.

[15] J Pradines, V Farutin, S Rowley, and V Dancik. Analyzing protein lists with large networks: edge-count probabilities in random graphs with given expected degrees. *Journal of Computational Biology*, 12(2):113–128, 2005.

[16] D Szklarczyk, R Kirsch, M Koutrouli, K Nastou, F Mehryary, R Hachilif, AL Gable, T Fang, N T Doncheva, S Pyysalo, et al. The string database in 2023: protein–protein association networks and functional enrichment analyses for any sequenced genome of interest. *Nucleic Acids Research*, 51(D1):D638–D646, 2023.